
Python for Everybody

Course 4:

USING DATABASES WITH PYTHON

MODULE 1:(OBJECT ORIENTED PYTHON)

- Python is an object oriented language

- *OOP'S four pillars:*

1. *Encapsulation:*

>>>wrapping data(variables) and methods (functions) together in a single unit(class).

>>>helps in data hiding(uses private variable)

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          # public
        self.__balance = balance    # private (encapsulation)

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

# Usage
account = BankAccount("Alice", 1000)
account.deposit(500)
print(account.get_balance()) # ✅ 1500
# print(account.__balance)   # ❌ Error: private
```

2. Abstraction:

>>>>Hiding implementation details and showing the essential features.

>>>>to be done using abstract classes and abstract method

- In this example @abstractmethod decorator was added in Vehicle class before start method
- It means start method declared within vehicle class is an abstract method.
- The classes that inherit from vehicle class has to provide their own specific way to implement start method
- And here polymorphism is also represented as different objects are responding to the same method call in their own way.

```
from abc import ABC, abstractmethod

class Vehicle(ABC): # abstract class
    @abstractmethod
    def start(self):
        pass

class Car(Vehicle):
    def start(self):
        print("Car engine started.")

class Bike(Vehicle):
    def start(self):
        print("Bike engine started.")

# Usage
v1 = Car()
v1.start() # Car engine started.
```

3. Inheritance:

>>>A class(child class)can inherit properties and methods from another class(parent class)

>>>>this is also promotes code reusability.

```
class Parent:
    def greet(self):
        print("Hello from the Parent class!")

class Child(Parent):
    pass # This means no additional features... yet!

# Create an object of the Child class
obj = Child()
obj.greet() # Output: Hello from the Parent class!
```

4. Polymorphism :

>>>allows same functions name ,but works differently on the object.

>>>usually achieved by method overriding and method overloading

```
class Bird:
    def sound(self):
        return "Tweet"

class Dog:
    def sound(self):
        return "Bark"

def make_sound(animal):
    print(animal.sound())

# Create objects
bird = Bird()
dog = Dog()

# Same function call, different behaviors
make_sound(bird) # Output: Tweet
make_sound(dog)  # Output: Bark
```

MODULE 2:(BASIC STRUCTURED QUERY LANGUAGE)

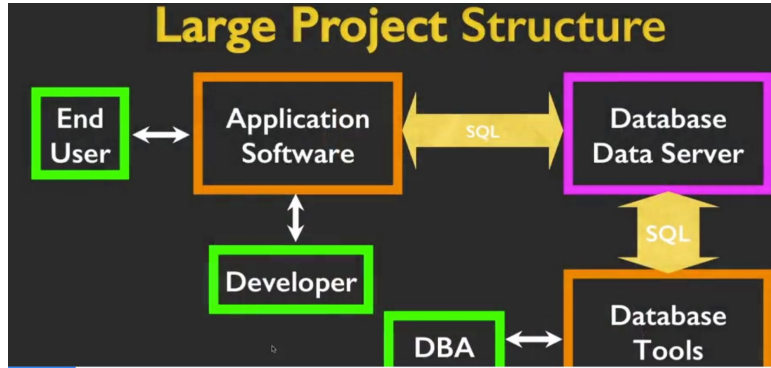
- **Relational database:**

- Relational databases model data by storing rows and columns in tables.
- It allows us to retrieve data from those tables and in particular where there are multiple tables and the relationships between those tables involved in the query
- terminologies:
 1. Database-collection of tables
 2. relation/table-contains tuples and attributes.
 3. Tuple - a set of fields that generally represents an “object” like a person or a music track
 4. Attribute-one of possible many elements of data corresponding to the object represented by rows.

- **SQL(STRUCTURED QUERY LANGUAGE):**

- It is a language we use to issue commands to the database
 1. Create a table
 2. Retrieve data
 3. Insert data
 4. Delete data

- **USING DATABASES:**



Two Roles in Large Projects

- **Application Developer** - Builds the logic for the application, the look and feel of the application - monitors the application for problems
- **Database Administrator** - Monitors and adjusts the database as the program runs in production
- Often both people participate in the building of the “Data model”

- **DATABASE ADMINISTRATOR(DBA):**

- A database administrator is a person responsible for the design, implementation, maintenance and repair of a organization database.
- The role includes the development and design of database strategies, monitoring and improving database performance.

- **DATABASE MODEL:**

- A database model or database schema is the structure or format of a database, described in a formal language supported by dbms.

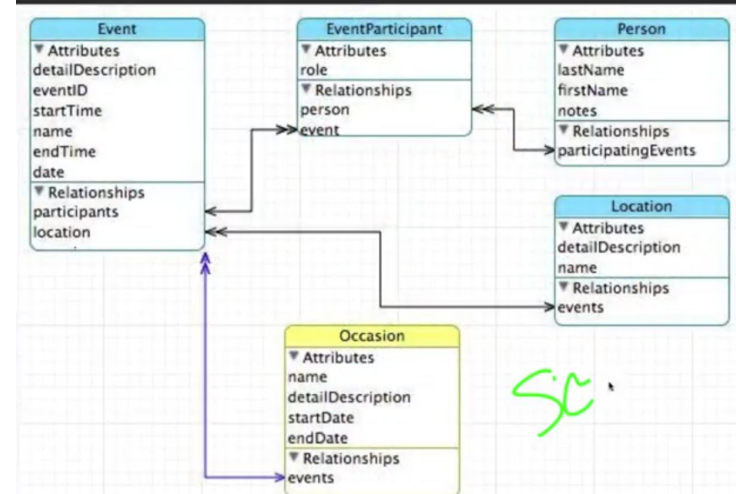
- Three Major Database Management Systems in wide use
 - **Oracle** - Large, commercial, enterprise-scale, very very tweakable
 - **MySQL** - Simpler but very fast and scalable - commercial open source
 - **SqlServer** - Very nice - from Microsoft (also Access)
- Many other smaller projects, free and open source
 - HSQL, **SQLite**, Postgress, ...

- **SQLITE BROWSER:**
- Sqlite is a very popular database
- It is free.
- We can do all types of sql command in sqlite:
 1. DDL
 2. DML
 3. TCL
 4. DCL
 5. DQL

MODULE 3:(DATA MODEL AND RELATIONAL SQL)

● DATABASE DESIGN:

- Database design is an art form of its own particular skills and experience.
 - Our goal is to avoid the really bad mistakes and design clean and easily understood databases.
 - Entity-relationship diagram:
 - An entity relationship diagram is a visual way to model the data in a system:
1. Entities: represents real world object.
 2. Attributes: properties of entity
 3. Relationship: connection between entities

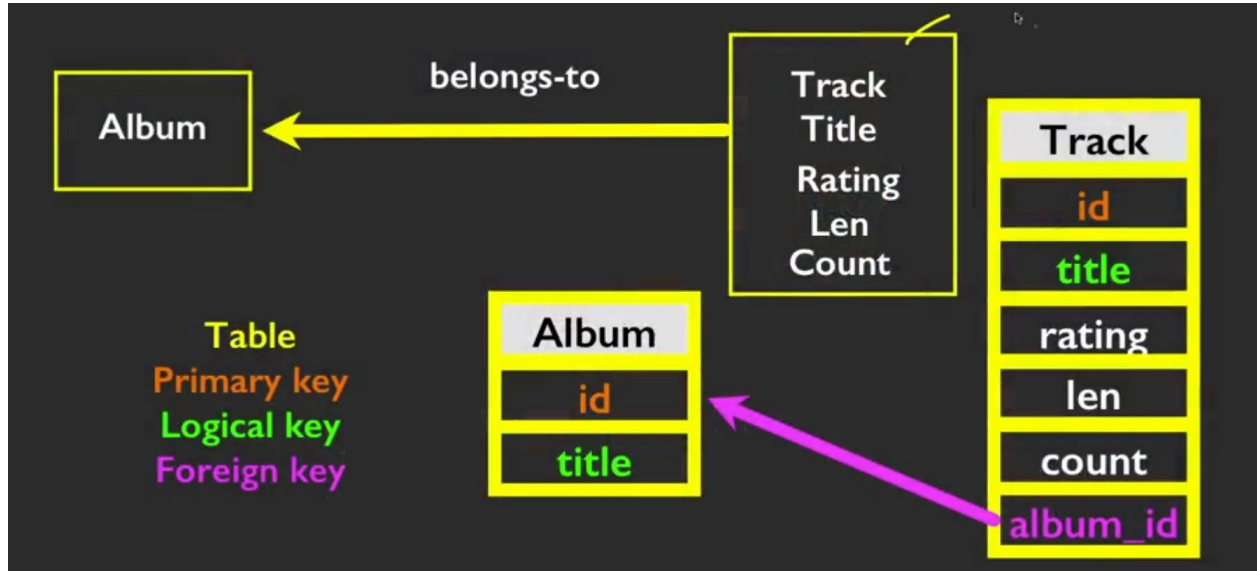


- **BUILDING A DATA MODEL:**

- Drawing a picture of the data objects for our application and then figuring out how to represent the objects and their relationships.
- Basic rule: Don't put the same string dat in twice-use a relationship instead.
- When there is one thing in the “real world” there should be one copy in the database as well.

- **REPRESENTING A DATA MODEL IN TABLES:**

- example:



- Inserting data in relational database is to be done by using insert into command.*
- ***JOIN OPERATION:***
- The join operation links across several tables as part of a select operation.
- We can multiple elements .

MODULE 4:(MANY-TO-MANY RELATIONSHIP)

- **MANY-TO-MANY:**
- A many-to-many relationship is a type of relationship in databases where multiple records in one table are associated with multiple records in another table.
- We need to add connection table with two foreign key.
- There is usually no separate primary key
- example:

```
-- Table 1: Students
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(100)
);

-- Table 2: Courses
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);

-- Junction Table: Student_Courses (for M:N relationship)
CREATE TABLE Student_Courses (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

- **ONE-TO-MANY and MANY-TO-ONE :**
- One record in Table A is related to many records in Table B
- But each record of table B relates back to only one record in table A
- Example:

```
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(100)  
);  
|  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(100),  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```

- department->employee is one-to-many
- And from employee->department it is many-to-one

MODULE 5:(DATABASE AND VISUALISATION)

- DATA MINING:
- Data mining is process of discovering a patterns and relationships ,trends and useful information from large dataset using statistical ,mathematical and computational techniques
- Majorly extracts hidden,previously unknown patterns.